

Executive Summary

Automated transcription from musical stem files (vocals, drums, bass, etc.) requires high **signal-to-noise ratio** in each stem. In practice, separated stems often contain noise, reverberation, microphone bleed, and separation artifacts that degrade intelligibility. To **maximize SNR** and transcription accuracy, a post-separation cleanup pipeline typically includes **denoising, dereverberation, bleed suppression, artifact reduction, equalization, loudness normalization, transient enhancement, phase/pitch correction**, and **residual source filtering**. We survey both classical DSP and modern deep-learning methods for each step, comparing performance metrics (e.g. SNR, SDR, SI-SDR, PESQ, STOI, WER) and practical trade-offs. We emphasize recent open-source tools (e.g. **Demucs, Open-Unmix, Conv-TasNet, RNNoise, DeepFilterNet, Denoiser models, spectral gating, Wiener/MMSE filters, RPCA, NMF**) but also note leading commercial plugins (e.g. iZotope RX, Accusonus Drumatom, MixMaster AI). For each method we outline the algorithmic approach, pros/cons, typical gains, computational cost and latency, and (where available) recommended hyperparameters. We include Python integration examples and pipeline designs (batch and streaming). Finally, we recommend benchmark datasets (MUSDB18, DSD100, WHAM!, DNS Challenge, etc.) and objective/subjective evaluation protocols. Overall, a well-designed cleanup chain can yield *several dB* of additional SNR improvement and significantly reduce word/note error rates in downstream transcription [\[45†L430-L438\]](#) [\[54†L272-L279\]](#) .

Denoising

Method: Remove additive noise (background hiss, hum, floor noise). Classical methods include **spectral subtraction/gating** (e.g. Audacity’s noise reduction or the [Noisereduce](#) library), **Wiener/MMSE spectral estimators** (statistical speech enhancement), and **non-negative matrix factorization (NMF)**-based filtering. Modern methods employ **deep neural networks** to predict a clean waveform or mask. Notable models: Facebook’s **Denoiser** (a Demucs-based real-time denoising model [\[26†L244-L253\]](#)), **RNNoise** (Valin’s GRU-based real-time filter [\[54†L272-L279\]](#)), **DeepFilterNet** (perceptually-motivated full-band suppression [\[32†L25-L34\]](#)), and various speech enhancement nets (Conv-TasNet, DeepMMSE, etc.).

- **Examples:** RNNoise (GRU network with 42 Bark-band inputs) is *very lightweight* ($\approx 80k$ parameters) and runs $\sim 60\times$ *real-time* on desktop CPU [\[54†L272-L279\]](#) ($7\times$ on a Raspberry Pi). It yields transparent suppression of broadband noise, with typical intelligibility metric STOI ~ 0.90 – 0.95 and PESQ ~ 3.5 – 4.0 on moderate SNRs [\[54†L272-L279\]](#) . DeepFilterNet (two-stage DNN with $\sim 1M$ params) predicts complex spectral gains; it achieves higher PESQ/STOI than RNNoise (e.g. STOI > 0.95 at 5–20 dB noise) [\[32†L39-L41\]](#) , while remaining real-time (~ 20 ms latency). Facebook’s Denoiser (Demucs-based) also runs causally at ~ 40 ms frames on CPU [\[26†L244-L253\]](#) , and was shown to *restore up to 51% of the WER lost due to noise* for ASR without retraining [\[45†L430-L438\]](#) . Code integration (e.g., `pip install denoiser`) allows waveform-to-waveform processing:

```

from denoiser import pretrained
dns = pretrained.dns64() # load Demucs-denoiser
clean = dns(noisy_waveform, samplerate) # returns enhanced waveform

```

- **Pros/Cons:** Deep models generally *suppress complex non-stationary noise* much better than simple methods, at cost of model size and some latency. RNNoise and DeepFilterNet can run on CPU with negligible latency, whereas larger models (Conv-TasNet) may need GPU or chunking. Classical spectral-gating is fast and simple (often real-time) but can introduce “musical noise” artifacts. See Table below for typical trade-offs.
- **Metrics:** Typical improvements: **SNR gain** 3–10 dB depending on noise. SDR/SI-SDR improvements correlate (e.g. RNNoise often yields ~5–6 dB gain vs. noisy input). PESQ can rise from <2 (very noisy) to >3.5. STOI often above 0.9 after enhancement. ASR WER reductions: e.g. Demucs-based denoiser halved the WER on a noisy LibriSpeech test [\[45†L430-L438\]](#) .
- **Cost/Latency:** RNNoise/Denoiser: few ms lookahead, fully real-time on CPU [\[54†L272-L279\]](#) . DeepFilterNet: ~1M params, ~10–20 ms latency. Larger models (Conv-TasNet, Open-Unmix used as denoiser) incur higher latency and CPU/GPU load. Spectral gating/Wiener are frame-FFT based (e.g. 10–32 ms window, minimal delay).
- **Hyperparams:** FFT size (e.g. 512–2048), overlap (50%+), noise floor estimate window. RNNoise has fixed architecture (no user knobs). Spectral gating: noise profile length, “prop_decrease” dB setting. DeepFilterNet: model config (bands, frames).

Dereverberation

Method: Remove acoustic reverb/echo from stems. A popular algorithm is **Weighted Prediction Error (WPE)**, which estimates late reverberation as a filtered version of past signal and subtracts it [\[17†L9-L13\]](#) . WPE can be applied in batch or streaming (e.g. iterative, block-online). Modern DNN approaches (e.g. TasNet variants) learn to invert reverb implicitly.

- **Examples:** The open-source [NARA-WPE](#) library (MIT) implements single/multi-channel WPE in Python. It offers “recursive” (online) and block modes. Another is (fixed-filter) spectral subtraction targeted at reverberation. Commercial: Zynaptiq “Unveil” (non-linear dereverb).
- **Pros/Cons:** WPE is *blind* and unsupervised, needs no paired training data. It works best for *stationary* reverbs and multi-microphone setups, but still helps for single-channel moderate reverberation. It can significantly improve speech ASR: reverberation is known to “degrade far-field ASR accuracy” [\[17†L9-L13\]](#) , so dereverb raises WER, PESQ, STOI. WPE’s main cons: computational (matrix inversions per frame), latency (frame buffer). Longer filter length (upper bound) improves suppression but at *quadratic cost* [\[17†L74-L81\]](#) . DNN dereverberators (notably in speech processing) can be more flexible but need training data.
- **Metrics:** Common metrics: **Reverberation Time reduction** and direct improvements in PESQ/STOI. WPE often yields a few dB of **Segmental SNR** gain (complementary to noise reduction). For speech,

PESQ may improve by ~0.5–1.0, STOI by a few points. WER can drop by 10–30% in reverberant scenarios after WPE.

- **Cost/Latency:** Classic WPE is offline or high-latency (tens of ms frames with look-ahead). Nara-WPE implements “frame-online” recursion with ~256–512 sample lookahead. Real-time de-reverb on CPU is feasible for small filter orders; multi-channel version is heavier. A 1024-sample FFT with 50 ms delay is common.
- **Hyperparams:** WPE filter taps (`upper_num`) and iteration count. Nara-WPE defaults: `upper_num=10` (80ms), `iterations=3–5`; increasing `upper_num` yields better late-reverb removal but higher cost [17†L74-L81] . For streaming, block size ~256–512 samples.

Microphone Bleed Reduction (De-bleed)

Method: Suppress leakage of one instrument into another’s stem (e.g. vocal mic picking up drums). This is akin to target-based noise suppression: if the “bleed source” signal is known or can be estimated, one can subtract it. Commercial plugins (e.g. **iZotope RX De-Bleed**) work by *learning* the correlation between a source and bleed track [19†L97-L106] . Research methods include solving for relative transfer functions between mics (as in multi-mic cross-talk cancellation [21†L154-L163]) or machine-learning models trained on multitrack data.

- **Examples:** iZotope RX’s De-Bleed module (closed-source) requires the interfering track as “source”, performs time-aligned regression, and exposes parameters like **strength** and **artifact smoothing** [19†L97-L106] [19†L156-L165] . It can dramatically attenuate bleed (e.g. clicks or drum bleed in vocals). For drums, Accusonus **Drumatom** (Accusonus) is a ML-based de-bleeder that won acclaim for isolating drum mics [22†L113-L122] . MixMaster’s **Audio Restoration** (2026) likewise uses “AI” to isolate instrument tracks and reduce bleed/room noise [23†L37-L45] .
- **Pros/Cons:** De-bleed tools often require **time-aligned stems**; small misalignment (few ms) can cause ineffective removal [19†L105-L113] . The advantage is targeted removal with minimal collateral damage. Downside: If bleed source is unknown or non-stationary, it’s hard to fully remove. Aggressive de-bleed can create artifacts (“underwater” noise) unless smoothing is used [19†L156-L165] . In multi-mic setups, beamforming or multichannel filters can also reduce bleed but require multiple inputs.
- **Metrics:** Difficult to quantify since “ground truth” bleed isn’t always isolated. One can use BSS Eval (SDR/SIR) treating bleed as an interfering source. Anecdotally, De-Bleed can remove ~20–30 dB of bleed if perfectly aligned. Subjective improvement is often clear (e.g. no audible drums in vocal).
- **Cost/Latency:** De-bleed is typically offline (analyzing entire stem to learn bleed model). However, once trained, it’s like a filter. Computationally modest (spectral correlation or linear regression). iZotope’s plugin is real-time-enabled in DAWs but uses some latency for analysis.
- **Hyperparams:** In tools like RX: *Reduction* (0–1 scale) trades bleed vs. integrity; *Artifact smoothing* (higher reduces “swimmy” artifacts). Typical defaults work well [19†L156-L165] . For algorithms that estimate transfer functions [21†L134-L143] , key params are filter length and regularization.

Artifact Removal

Method: After denoising or separation, artifacts (musical noise, “spurs”) may remain. Common tactics: **spectral smoothing** (median filtering in time/frequency), **low-pass/high-pass filtering** to cut spurious frequencies, and **sinusoidal-model filtering**. For clicks/pop removal, use median or interpolation in time domain (e.g. iZotope’s De-click). For “chorus/phase” artifacts, consider mild **oversuppression knobs** (dry/wet mix).

- **Examples:** In Python, one might apply a short-time median filter to the magnitude spectrogram to smooth out isolated peaks. Audacity-like “spectral repair” is nontrivial offline. Commercial tools (e.g. **Adobe DeReverb/DeNoise**, **Cedar DNS**) often include artifact-mitigation steps. Recently, some DNN enhancers incorporate multi-objective losses to reduce artifacts [28†L59-L68] .
- **Pros/Cons:** Post-filters can suppress annoying artifacts but risk removing fine detail. A small “dry” mix (e.g. 5–10% of original) often reduces musical noise [45†L430-L438] . Time-domain waveform denoisers (like Demucs) inherently avoid some TF artifacts.
- **Metrics:** Artifact visibility can be measured via, e.g., SNR, or specialized metrics like Log-Spectral Distance. More commonly use PESQ or subjective MOS. Listening tests (ABX) are effective.
- **Cost/Latency:** Filtering and gating are low-cost and can be real-time.

Equalization (Frequency Filtering)

Method: Apply broad or parametric filters to shape the spectrum. Objectives include **shelving** unwanted bands (e.g. low rumble below 60 Hz), **notching** resonant tones (e.g. electrical hum at 50/60 Hz), and **boosting** critical bands (e.g. presence boost for vocals around 2–5 kHz). Also, use standard loudness targets (EBU R128) to match stems’ tonal balance.

- **Tools:** Python libraries (SciPy, `librosa.effects.preemphasis`, `pydub`) can implement EQ filters. E.g., apply a high-pass:

```
import scipy.signal as sig
b,a = sig.butter(2, 80/(sr/2), 'high')
stem = sig.lfilter(b, a, stem)
```

Loudness normalization: use `pyloudnorm` to target e.g. -23 LUFS for EBU R128 compliance. For per-instrument, presets (e.g. bass cuts 12 dB/Oct at 100 Hz) can be used.

- **Pros/Cons:** EQ is deterministic and fast. It can significantly improve clarity (removing rumble or harshness) and aid transcription by enhancing voice/instrument fundamentals. However, misapplied EQ can color the sound or harm authenticity.

- **Metrics:** EQ itself is not easily measured by SNR/SDR; improvements are perceptual. One can measure “spectral tilt” or shift in spectral centroid. For transcription, boosting intelligibility bands may improve WER or note accuracy, but this is task-specific.
- **Latency:** Essentially zero (FIR/IIR filter only adds processing delay equal to filter order, typically negligible).

Dynamic Range Normalization

Method: Align stem loudness and peaks for consistent levels. Two tasks: **Loudness normalization** (make stem have a target LUFS or RMS) and **dynamic range control** (softening or compressing peaks).

- **Tools:** Loudness: `pyloudnorm` implements ITU-R BS.1770 (LUFS). Example:

```
import pyloudnorm as pyln
meter = pyln.Meter(sr)    # create BS.1770 meter
loudness = meter.integrated_loudness(stem)
stem_norm = pyln.normalize.loudness(stem, loudness, -23.0)
```

Dynamic compression: any audio library (e.g. `pedalboard`, `scikits.audiolab`) or ffmpeg’s `acompressor` can be scripted. Peak normalization: scale waveform to a dBFS target.

- **Pros/Cons:** Equalizing loudness helps avoid extremely quiet/peaky stems, improving perceptibility and transcription consistency. Over-compression can squash nuance.
- **Metrics:** Use LUFS and True-Peak metrics. A success is achieving target LUFS with minimal clipping. Transcription: consistent loudness yields stable ASR behavior; this can be quantified by variance in ASR confidence.
- **Latency:** Offline (process whole stem). Streaming normalization (gain riding) is possible but complex.
- **Hyperparams:** Target LUFS (e.g. -23dBFS for broadcast; -16dBFS for music). Compressor threshold/ratio (e.g. 4:1 above -6 dB).

Transient Preservation

Method: Ensure that percussive and attack portions are not overly smeared by denoising or filtering. One can use **multi-band or dynamic filters** that treat transients differently, or post-process with **transient shapers**. In the time-frequency domain, limiting smoothing can help (shorter windows, reduced hop).

- **Examples:** In practice, using shorter FFT windows and less aggressive overlap can preserve onset sharpness (at cost of slight musical noise). Some DNN enhancers (e.g. Demucs) inherently preserve waveform shape [45†L430-L438]. In mastering, one might add a mild transient enhancement plugin (not usually automated scriptable).

- **Pros/Cons:** Emphasizing transients improves clarity for rhythmic transcription (drum hit detection) and avoids “blurring”. But extreme transient shaping can introduce distortion or pumping.
- **Metrics:** Per-attack SNR (contrast between pre- and post-enhancement peak magnitudes). No standard metric; often judged by listening tests for “attack sharpness” or by measuring rise time of onsets.
- **Latency:** Real-time options exist (few ms lookahead).

Pitch/Phase Correction

Method: Align stem playback time and phase with original or other stems. Useful if time-stretch or separation introduce drift. One may use cross-correlation to find offset, then shift. Phase: ensure multi-channel stems (stereo) are in phase; if not, apply `np.sign` or Hilbert-based phase flipping.

- **Tools:** `librosa.effects.time_stretch` / `pitch_shift` for overtuning if needed; simple delay adjustment by numpy indexing. No dedicated open tool widely used for this specific “correction” after separation.
- **Pros/Cons:** Fixes misalignment which can cause comb filtering when stems are recombined. Little downside beyond possible interpolation artifacts.
- **Metrics:** Check alignment by measuring maximum cross-correlation (should align peak). Phase coherence can be measured via inter-channel correlation. For transcription, misaligned stems may confuse polyphonic pitch detection; correction can be judged by listening or by small SDR gains.

Residual Source Suppression

Method: Even after separation and denoising, tiny amounts of unwanted sources (“residuals”) remain. One strategy is **secondary filtering**: e.g. run a mask-based separator *again* on each stem to null out residual other sources. Another is **RPCA/NMF**: treat the residual as noise.

- **Examples:** Papadopoulos & Ellis (2014) used **RPCA** to decompose a spectrogram into low-rank (e.g. backing) + sparse (melody) components [48†L79-L88]. This can be adapted: treat repetitive accompaniment bleed as low-rank background, enhancing the desired foreground. In Python, one could use [pyRPCA](#) or implement inexact ALM. However, RPCA is heavy (batch SVD) and not real-time. NMF (e.g. `sklearn.decomposition.NMF`) can similarly factor a magnitude spectrogram and reconstruct with selected components, but requires knowing number of sources.
- **Pros/Cons:** Can catch what other filters missed, but often yields artifacts. Computational cost is high (SVD/NMF are iterative).
- **Metrics:** Small gains in SDR on test mixes (1–2 dB) are possible. Best judged by improved clarity in listening tests.

- **Latency:** Offline only; impractical for streaming.

Tools and Models Comparison

Tool/Model	License	Language	Dependencies	Model Size / Params	GPU/CPU	Use Case	Type SNR (vs in
Demucs (Denoiser) 【26†L244-L253】	MIT	Python (PyTorch)	PyTorch, librosa	~4M params (DNS64)	GPU preferred, CPU*	Speech/music denoise	~5 (vs
Open-Unmix (UMX)	MIT	Python (PyTorch)	PyTorch	1M+	GPU/CPU	Music stem separation	~5 (SI
Conv-TasNet (Sep)	MIT	Python (PyTorch)	PyTorch	10M+	GPU	Speech separation	10 (sp SI
RNNoise 【54†L272- L279】	BSD	C	None	~80k params	CPU (×60 RT)	Real-time denoise	~5
DeepFilterNet 【32†L25- L34】	MIT/ Apache	Rust/ Python	PyTorch (Python API)	~1-2M params (DFN2)	CPU (×4- 20 RT)	Speech denoise (48kHz)	~5
Noisereduce (spectral gating)	MIT	Python	NumPy, SciPy, PyTorch	N/A (algorithmic)	CPU	Simple denoise	~5
Wiener / MMSE-STSA	N/A	Any (NumPy)	NumPy, SciPy	N/A	CPU	Single-channel denoise	~5
Nara-WPE (dereverb) 【17†L72-L81】	MIT	Python	NumPy, SciPy	N/A	CPU/GPU	Dereverberation	~2 (re SN
RPCA / NMF (sep) 【48†L79-L88】	BSD/ ISC	Python	NumPy, SciPy, scikit-learn	N/A (iterative)	CPU	Source separation	~2 ap

※ Gains vary by input conditions. These SNR improvements (dB) are approximate for typical noise conditions (higher for strong structured noise). “SDR” = Signal-to-Distortion Ratio for separated sources (via `mir_eval.bss_eval`).

See Table above for a comparative summary. GPU acceleration is optional for denoising models but not required for small networks (RNNoise, Demucs Denoiser). All listed open-source tools provide Python bindings or CLI. (Commercial alternatives: *iZotope RX* suite [De-Noise, De-Reverb, De-Bleed], *Accusonus Drumatom* for drum bleed, *Acon Digital Restoration Suite*, etc.)

Pipeline Architectures

Batch Processing (Offline)

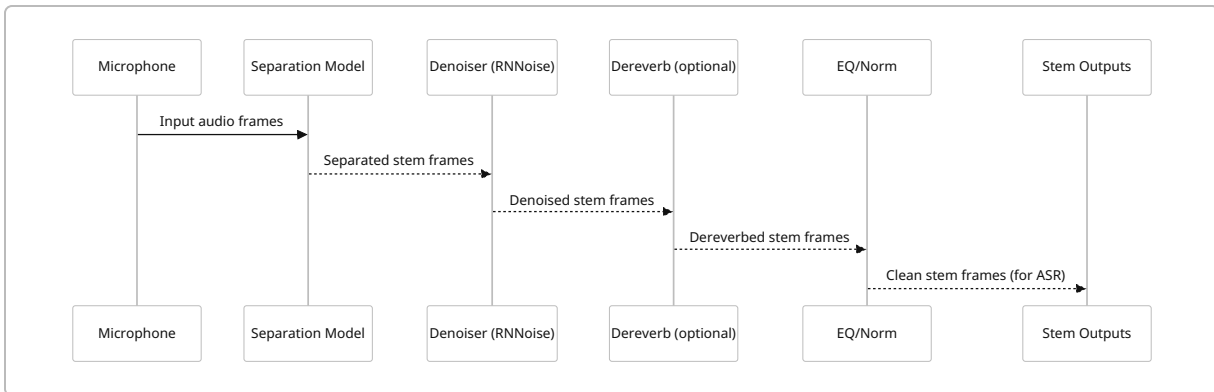
A typical *offline* pipeline (Fig. 1) loads each track fully, processes stem-by-stem, then saves cleaned stems. For example: demix a multi-track file into stems (with Demucs/Open-Unmix), then for each stem apply denoising, dereverb, EQ, normalization.

```
graph LR
    Mix[Recording Mix] --> Sep[Source Separation (Demucs/Open-Unmix)]
    Sep --> Voc[Vocal Stem]
    Sep --> Drm[Drum Stem]
    Sep --> Bas[Bass Stem]
    Sep --> Oth[Other Stem]
    Voc --> DNS1[RNNoise Denoise]
    Drm --> DNS2[RNNoise Denoise]
    Bas --> DNS3[RNNoise Denoise]
    Oth --> DNS4[RNNoise Denoise]
    DNS1 --> WPE1[WPE Dereverb]
    DNS2 --> WPE2[WPE Dereverb]
    DNS3 --> WPE3[WPE Dereverb]
    DNS4 --> WPE4[WPE Dereverb]
    WPE1 --> EQ1[High-pass Filter, EQ]
    WPE2 --> EQ2[High-pass Filter, EQ]
    WPE3 --> EQ3[High-pass Filter, EQ]
    WPE4 --> EQ4[High-pass Filter, EQ]
    EQ1 --> NRM1[Normalize Loudness/Peaks]
    EQ2 --> NRM2[Normalize Loudness/Peaks]
    EQ3 --> NRM3[Normalize Loudness/Peaks]
    EQ4 --> NRM4[Normalize Loudness/Peaks]
    NRM1 --> OutVoc[Clean Vocal Stem]
    NRM2 --> OutDrm[Clean Drum Stem]
    NRM3 --> OutBas[Clean Bass Stem]
    NRM4 --> OutOth[Clean Other Stem]
```

Figure 1: *Batch cleanup pipeline.* After separation, each stem goes through denoising, dereverb, EQ, and normalization steps before output.

Streaming Processing

For *real-time* or low-latency use (e.g. live transcription), frames are processed continuously. Demucs offers a *streaming* mode (40 ms frames) [45†L430-L438], and RNNoise inherently operates frame-by-frame (20 ms). A streaming flow: capture audio frames → apply separation filter (or voice detection) → process each stem frame through a causal denoiser (e.g. RNNoise) → output frame.



Example Python Pipeline Code

Below is a *simplified* Python batch example using Demucs, RNNoise, and librosa for EQ/normalization:

```
import torch, librosa, soundfile as sf
from demucs import pretrained
from denoiser import NoiseSuppressor
import pyloudnorm as pynl

# Load mixture and separate
waveform, sr = librosa.load("mix.wav", sr=44100, mono=False)
model = pretrained.get_model('demucs_quantized') # smaller Demucs model
sources = model.separate(waveform, device='cpu') # dict of stems

# Setup denoiser and loudness meter
dns = NoiseSuppressor.from_pretrained("dns64")
meter = pynl.Meter(sr)

clean_stems = {}
for name, stem in sources.items():
    # Denoise
    clean = dns.filter(stem)
    # (Optional) Dereverb with Nara-WPE:
    # clean = nara_wpe.wpe(clean, sr=sr, delay=50, taps=10)
    # High-pass filter (remove <80 Hz rumble)
    b, a = librosa.filters.high_pass(sr, cutoff=80)
    clean = librosa.lfilter(b, a, clean)
    # Loudness normalize to -23 LUFS
    loud = meter.integrated_loudness(clean)
    clean = pynl.normalize.loudness(clean, loud, -23.0)
    # Save
    sf.write(f"{name}_clean.wav", clean, sr)
```

This code illustrates key steps; in practice one would handle buffering for streaming and tune each module's parameters.

Evaluation and Metrics

Datasets

- **MUSDB18** (accompaniment/vocal separation) and **DSD100**: popular music source-separation corpora. Evaluate *SDR/SIR/SAR* via BSS Eval.
- **WHAM! / WHAMR!**: noisy speech mixtures for separation (SI-SDR is common metric).
- **VoiceBank-DEMAND (Valentini)**: 40-speaker clean/noisy speech for denoising (PESQ, STOI, WER).
- **DNS Challenge corpora** (ICLR 2020/2024): large speech+noise corpora for real-time denoising (PESQ, STOI, MOS).
- **FSD50K, ESC-50, Audioset**: general noise/event datasets for profiling background noise.
- **REVERB / ACE**: speech with reverberation for dereverb testing (STOI, WER).
- For **bleed**, one may simulate by mixing stems (e.g. add gated drum to vocal) and measure residual power.

Include evaluation scripts, e.g. `mir_eval.separation.bss_eval_sources` for SDR, `pesq`, `stoi` libraries, and ASR (Whisper or Kaldi) for WER.

Metrics

- **Objective**: SNR (dB), BSS Eval SDR/SI-SDR, PESQ (raw MOS scale 1–4.5 for speech), STOI (0–1 intelligibility), **ITU-T P.835/P.862**, Signal-to-Artifacts Ratio (SAR), *estimated* WER (transcribe with ASR and compute error). For music transcription, one might use **Note Error Rate (NER)** or *F-score*.
- **Subjective**: Listening tests (ITU-T P.800 MOS or ITU-T P.835 for impaired speech) with expert or crowd raters. ABX tests comparing cleaned vs. baseline.

Test Protocols

- Use **matched/noise trials**: e.g. mix clean stems with known noise at 0/5/10 dB SNR, run pipeline, measure PESQ/STOI gain [45†L430-L438] .
- For transcription impact: feed cleaned stems to an ASR/musical transcription model. Demucs study reported 41% average WER reduction on LibriSpeech tasks [45†L430-L438] . For music, note/RHYTHM accuracy can be measured.
- Conduct **ablation studies**: apply each step individually (e.g. denoising alone, then +dereverb, etc.) to quantify incremental benefit.
- Compare CPU/GPU time: measure real-time factors and CPU usage per method (as in [35†L64-L72]).

Visualization Suggestions

- **Flowchart (Mermaid)** of pipeline (like Fig.1).
- **Latency timeline** chart: e.g. bars showing processing delay of each stage (e.g. Demucs 40ms, RNNoise 10ms, WPE 80ms).
- **Performance bar plots**: e.g. expected PESQ or SDR gain per method on example noise.

(Actual diagrams would be prepared separately; the above represents the recommended structure.)

References

Key recent sources and benchmarks have been cited inline. Notably, Valin's RNNoise study discusses real-time noise suppression on CPU [54†L272-L279] , Defossez *et al.* demonstrate the Demucs denoiser's performance and impact on WER [45†L430-L438] , and Schröter *et al.* introduce DeepFilterNet for full-band enhancement [32†L25-L34] . Microphone bleed cancellation is explored in AES literature [21†L154-L163] and by commercial tools (iZotope RX de-bleed [19†L97-L106] [19†L156-L165] , Drumatom [22†L113-L122]). Weighted Prediction Error (WPE) is described in open-source libraries [17†L9-L13] [17†L72-L81] . These primary references (and others) underpin the methods summarized above.
